

MARAN PMS — Fixes Reservas

PROMPT: Superposición de fechas + Reservas cerradas + Historial de cambios

PROBLEMA 1 — Superposición de fechas no se muestra al usuario

Diagnóstico

El backend ya tiene `checkOverbooking()` y devuelve status 409 con el mensaje correcto. El problema está en el frontend: la mayoría de los `onError` de las mutations de reserva muestran un mensaje genérico e ignoran el mensaje real del servidor.

Fix — Estandarizar el parseo de errores en TODAS las mutations de reserva

Crear una función helper reutilizable:

```
// En el archivo donde viven las mutations de reserva (o en un utils compartido)
function parseReservationError(error: any): string {
  try {
    const raw = error?.message || "";
    const jsonStart = raw.indexOf("{");
    if (jsonStart >= 0) {
      const parsed = JSON.parse(raw.substring(jsonStart));
      return parsed.error || parsed.message || "Error desconocido";
    }
  } catch {}
  return "No se pudo completar la operación. Intente nuevamente.";
}
```

Aplicar esta función en los `onError` de las siguientes mutations:

1. Walk-in (check-in rápido desde recepción):

```
onError: (error: any) => {
```

```
toast({
  title: "Error",
  description: parseReservationError(error),
  variant: "destructive",
});
},
```

2. Nueva reserva (new-reservation.tsx y todos los formularios de creación):

```
onError: (error: any) => {
  toast({
    title: "No se pudo crear la reserva",
    description: parseReservationError(error),
    variant: "destructive",
  });
},
```

3. Edición de reserva (PATCH /api/reservations/:id):

```
onError: (error: any) => {
  toast({
    title: "No se pudo modificar la reserva",
    description: parseReservationError(error),
    variant: "destructive",
  });
},
```

El mensaje que llegará del servidor cuando haya superposición es: "La habitación 301 ya tiene una reserva en esas fechas."

Ese texto debe mostrarse tal cual en el toast, en rojo (variant: "destructive").

PROBLEMA 2 — Se puede editar reservas con estado checked_out

Diagnóstico

`isReservationLocked()` existe en el backend y bloquea correctamente. Pero el frontend sigue mostrando el formulario de edición habilitado para reservas `checked_out` y `cancelled`.

Fix A — Deshabilitar el botón/acceso de edición en el frontend

En todos los lugares donde se muestra una reserva y hay un botón de editar:

```
// Definir esta constante apenas se carga la reserva
const isLocked = reservation.status === "checked_out" || reservation.status === "

// Botón de editar:
<Button
  variant="outline"
  disabled={isLocked}
  title={isLocked ? "No se puede editar una reserva cerrada" : "Editar reserva"}
  onClick={() => setEditOpen(true)}
>
  <Pencil className="h-4 w-4" />
  Editar
</Button>
```

Fix B — Si el dialog de edición se abre igualmente, mostrar banner de aviso

```
{isLocked && (
  <div className="flex items-center gap-2 rounded-md bg-amber-50 border border-am
    <Lock className="h-4 w-4 text-amber-600 shrink-0" />
    <span className="text-sm text-amber-800">
      Esta reserva está cerrada y no puede modificarse.
    </span>
  </div>
)}
```

Y deshabilitar todos los inputs del formulario cuando `isLocked === true` :

```
<Input disabled={isLocked} ... />
<Select disabled={isLocked} ... />
<Textarea disabled={isLocked} ... />
// etc.
```

Fix C — Deshabilitar también desde el Planning

En el planning, cuando se hace clic en una reserva `checked_out` , no debe ofrecer la opción de editar. Solo mostrar el detalle en modo lectura.

PROBLEMA 3 — Historial automático de cambios en reservas

Nueva tabla en `shared/schema.ts`

```
export const reservationChangelog = pgTable("reservation_changelog", {
  id: serial("id").primaryKey(),
  reservationId: varchar("reservation_id").notNull().references(() => reservation),
  fecha: timestamp("fecha").defaultNow().notNull(),
  operador: text("operador"), // nombre del usuario que hizo el cambio
  tipo: text("tipo").notNull(),
  // Valores posibles:
  // "fecha" → cambio de check-in, check-out o noches
  // "habitacion" → cambio de habitación (camaje)
  // "estado" → cambio de estado (confirmed, checked_in, etc.)
  // "tarifa" → cambio de tarifa o descuento
  // "huesped" → cambio de huésped titular
  // "notas" → modificación del campo notas
  // "otro" → cualquier otro campo
  descripcion: text("descripcion").notNull(),
  // Texto legible, ej: "Fechas modificadas: 12/03 → 14/03 al 15/03 → 17/03"
  // ej: "Habitación cambiada: 301 → 405"
  // ej: "Estado: Confirmada → Check-in realizado"
});
```

Migrar con `npx drizzle-kit push`.

Backend — Detectar y registrar cambios automáticamente

En el handler `PATCH /api/reservations/:id`, **antes de llamar a** `updateReservation()`, **comparar los campos que cambiaron y generar los registros de changelog:**

```
app.patch("/api/reservations/:id", async (req, res) => {
  // ... validaciones existentes ...

  const existing = await storage.getReservation(req.params.id);

  // Detectar cambios y generar descripciones
  const cambios: { tipo: string; descripcion: string }[] = [];

  // Cambio de fechas
  if (req.body.checkInDate && req.body.checkInDate !== existing.checkInDate) {
    const oldIn = formatDate(existing.checkInDate); // DD/MM/YYYY
    const newIn = formatDate(req.body.checkInDate);
```

```

    cambios.push({ tipo: "fecha", descripcion: `Check-in modificado: ${oldIn} → $
}
if (req.body.checkOutDate && req.body.checkOutDate !== existing.checkOutDate) {
    const oldOut = formatDate(existing.checkOutDate);
    const newOut = formatDate(req.body.checkOutDate);
    cambios.push({ tipo: "fecha", descripcion: `Check-out modificado: ${oldOut} →
}

// Cambio de habitación
if (req.body.roomId && req.body.roomId !== existing.roomId) {
    const oldRoom = await storage.getRoom(existing.roomId);
    const newRoom = await storage.getRoom(req.body.roomId);
    cambios.push({
        tipo: "habitacion",
        descripcion: `Habitación cambiada: ${oldRoom?.roomNumber || existing.roomId
    });
}

// Cambio de estado
if (req.body.status && req.body.status !== existing.status) {
    const statusLabels: Record<string, string> = {
        tentative: "Tentativa",
        pending: "Pendiente",
        confirmed: "Confirmada",
        checked_in: "Check-in realizado",
        checked_out: "Check-out realizado",
        cancelled: "Cancelada",
    };
    cambios.push({
        tipo: "estado",
        descripcion: `Estado: ${statusLabels[existing.status] || existing.status} →
    });
}

// Cambio de tarifa
if (req.body.baseRatePerNight && req.body.baseRatePerNight !== existing.baseRat
    cambios.push({
        tipo: "tarifa",
        descripcion: `Tarifa modificada: $$${existing.baseRatePerNight} → $$${req.bod
    });
}

// Actualizar la reserva
const reservation = await storage.updateReservation(req.params.id, req.body);

// Guardar los cambios detectados
const operador = (req as any).user?.fullName || (req as any).user?.username ||

```

```

    for (const cambio of cambios) {
      await db.insert(reservationChangelog).values({
        reservationId: req.params.id,
        operador,
        tipo: cambio.tipo,
        descripcion: cambio.descripcion,
      });
    }

    res.json(reservation);
  });
}

```

Nuevo endpoint para obtener el historial

```

app.get("/api/reservations/:id/changelog", async (req, res) => {
  const logs = await db
    .select()
    .from(reservationChangelog)
    .where(eq(reservationChangelog.reservationId, req.params.id))
    .orderBy(asc(reservationChangelog.fecha));
  res.json(logs);
});

```

Frontend — Mostrar historial en el detalle de la reserva

En el componente de detalle/edición de reserva, agregar una sección al final:

```

// Query del historial
const { data: changelog = [] } = useQuery({
  queryKey: [`/api/reservations/${reservation.id}/changelog`],
});

// Render (debajo del formulario, siempre visible aunque isLocked)
{changelog.length > 0 && (
  <div className="mt-6">
    <h4 className="text-sm font-semibold text-muted-foreground uppercase tracking
      Historial de cambios
    </h4>
    <div className="space-y-2">
      {changelog.map((entry) => (
        <div key={entry.id} className="flex items-start gap-3 text-sm">
          <span className="text-muted-foreground shrink-0 w-32">
            {format(new Date(entry.fecha), "dd/MM/yyyy HH:mm")}
          </span>
          <span className="text-muted-foreground shrink-0 w-24">

```

```

        {entry.operador || "Sistema"}
      </span>
      <span className="text-foreground">{entry.descripcion}</span>
    </div>
  )))
</div>
</div>
)}

```

Los registros del historial **no tienen botón de editar ni eliminar**. Son de solo lectura.

PARTE 4 — Historial en el Cierre de Caja

En el cierre de caja diario, al final del resumen (donde ya aparecen los check-outs del día), agregar una sección con los cambios de reservas del día.

Nuevo endpoint

```

app.get("/api/reservations/changelog/hoy", async (req, res) => {
  const hoy = new Date();
  const inicioDia = startOfDay(hoy);
  const finDia = endOfDay(hoy);

  const logs = await db
    .select({
      id: reservationChangelog.id,
      fecha: reservationChangelog.fecha,
      operador: reservationChangelog.operador,
      tipo: reservationChangelog.tipo,
      descripcion: reservationChangelog.descripcion,
      reservationCode: reservations.reservationCode,
      guestFirstName: guests.firstName,
      guestLastName: guests.lastName,
    })
    .from(reservationChangelog)
    .innerJoin(reservations, eq(reservationChangelog.reservationId, reservations.
    .innerJoin(guests, eq(reservations.guestId, guests.id))
    .where(
      and(
        gte(reservationChangelog.fecha, inicioDia),
        lte(reservationChangelog.fecha, finDia)
      )
    )
    .orderBy(asc(reservationChangelog.fecha));

```

```
res.json(logs);  
});
```

En el componente de Cierre de Caja

Agregar al final del resumen del cierre, antes del botón de confirmar:

```
{changelogHoy.length > 0 && (  
  <div className="mt-6 border-t pt-4">  
    <h4 className="font-semibold mb-3 flex items-center gap-2">  
      <ClipboardList className="h-4 w-4" />  
      Modificaciones de reservas del día  
    </h4>  
    <div className="space-y-1 text-sm">  
      {changelogHoy.map((entry) => (  
        <div key={entry.id} className="flex items-start gap-3 py-1 border-b borde  
          <span className="text-muted-foreground w-12 shrink-0">  
            {format(new Date(entry.fecha), "HH:mm")}  
          </span>  
          <span className="text-muted-foreground w-28 shrink-0">  
            {entry.guestFirstName} {entry.guestLastName}  
          </span>  
          <span className="text-muted-foreground w-24 shrink-0">  
            [{entry.reservationCode}]  
          </span>  
          <span>{entry.descripcion}</span>  
          <span className="text-muted-foreground ml-auto shrink-0">  
            {entry.operador}  
          </span>  
        </div>  
      ) )}  
    </div>  
  </div>  
)}
```

Este bloque también debe incluirse en el **PDF del cierre de caja** si se genera uno.

NOTAS

1. La función `formatDate(dateStr)` convierte "YYYY-MM-DD" a "DD/MM/YYYY" para que los mensajes sean legibles en el historial.
2. El historial **no registra cambios de campos menores** como `bedTypeNotes` o

`externalReservationId` . Solo los campos relevantes operativamente: fechas, habitación, estado, tarifa, huésped.

3. Si una sola edición cambia múltiples campos (ej: fechas + habitación), se generan **múltiples filas** en `reservation_changelog` , una por cada cambio. Así el historial es granular.
4. No modificar el campo `notes` de la reserva para el historial. El historial usa la tabla nueva `reservation_changelog` , separada de las notas manuales del operador.